HTML exercises

Instructions

Use VS Code and a browser for these exercises.

For exercises that require a complete page:

- Create the file using the exact name shown.
- Save it before opening or refreshing the browser.
- Use Live Server where available.
- Do not add CSS, Tailwind CSS, or JavaScript.
- Keep the focus on HTML structure and meaning.

Complete each exercise independently unless it refers to an earlier part of the same exercise.

Section A: Document structure and content hierarchy

Exercise 1: Complete the HTML document

Create this file:

```
exercise-01/
└── index.html
```

The following content is missing its document structure:

```
<h1>Community reading room</h1>

<p>
  The reading room is open from
  9 AM to 6 PM.
</p>
```

Turn it into a complete HTML document.

Your page should include:

- `<!DOCTYPE html>`
- The root `<html>` element
- A `<head>` section
- UTF-8 character encoding
- A viewport meta element

- The browser-tab title `Community reading room`
- A `<body>` containing the supplied heading and paragraph

Do not place visible page content inside `<head>`.

Exercise 2: Separate document information from visible content

The following elements have been placed in the wrong parts of the document:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <h1>Weekend learning centre</h1>

  <p>
    Workshops begin at 10 AM.
  </p>
</head>
<body>
  <title>Weekend learning centre</title>

  <meta charset="UTF-8">

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >
</body>
</html>
```

Move each element to the correct location.

After correcting the page, answer:

1. Which information belongs in `<head>`?
2. Which content belongs in `<body>`?
3. Why is `<head>` different from a visible page header?

Exercise 3: Repair the nesting

The following document contains incorrectly nested and unclosed elements:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
  <title>Workshop information</title>
</head>
<body>
  <main>
    <p>
      <h1>Frontend workshop</h1>
      Learn the foundations of web development.
    </p>

    <section>
      <h2>Topics</h2>

      <ul>
        <li>HTML
        <li>CSS</li>
        <li>JavaScript</li>
      </ul>
  </main>
    </section>
</body>
</html>
```

Correct the structure without changing the visible text.

Your solution should:

- Keep the main heading outside the paragraph
- Close every list item
- Close nested elements in the correct order
- Keep all visible content inside `<body>`

Exercise 4: Create a meaningful heading structure

A course page contains:

- The page title: `Frontend foundations`
- A main section: `Course overview`
- A second main section: `Weekly schedule`
- A smaller topic inside the schedule: `Saturday session`
- Another smaller topic inside the schedule: `Sunday session`

Write suitable headings for this structure.

Apply these rules:

- Use one `<h1>` for the page title.
- Use `<h2>` for the two main sections.
- Use `<h3>` for the two sessions.
- Do not choose heading levels based only on the default text size.

Then add one short paragraph below each session heading.

Section B: Text, grouping, links, images, lists, and media

Exercise 5: Choose between `div` and `span`

Start with this content:

```
Frontend development combines HTML, CSS,
and JavaScript.

Course status: Enrolment open
```

Create HTML that meets these requirements:

- Place the paragraph and status inside one larger grouping element.
- Wrap only the words `HTML`, `CSS`, and `JavaScript` in inline grouping elements.
- Wrap only the words `Enrolment open` in an inline grouping element.
- Do not place a block of page content inside a `<span>`.

After writing the markup, explain:

1. Why the outer grouping uses a `<div>`
2. Why the words inside the sentences use `<span>`

Exercise 6: Complete a link and image

Complete the missing attributes:

```
<a ___="https://developer.mozilla.org/">
  Read the HTML documentation
</a>

<img
  ___="./images/workshop-room.jpg"
  ___="Learners working together in a computer room"
>
```

Use the correct attributes for:

- The link destination
- The image source
- The image alternative text

Then answer:

1. What happens when an `<a>` element has no `href`?
2. What may appear when the image path is incorrect?
3. What information should useful alternative text provide?

Exercise 7: Choose the correct type of list

Create HTML for both groups of information.

Workshop topics

The order does not matter:

```
HTML structure
CSS styling
JavaScript basics
```

Registration steps

The order matters:

```
Choose a workshop
Complete the registration form
Check the confirmation message
```

Use:

- An unordered list for the workshop topics
- An ordered list for the registration steps
- One `<li>` for each item

After writing the lists, explain why the two groups use different list elements.

Exercise 8: Add audio and video

Create this file:

```
exercise-08/
└── index.html
```

Build a complete HTML page titled `Workshop media`.

The page should contain:

Audio introduction

Add an audio player using:

```
./media/workshop-introduction.mp3
```

Include:

- The `controls` attribute
- A `<source>` element
- The media type `audio/mpeg`
- Fallback text for browsers that cannot play the audio

Video preview

Add a video player using:

```
./media/workshop-preview.mp4
```

Include:

- The `controls` attribute
- A `<source>` element
- The media type `video/mp4`
- Fallback text for browsers that cannot play the video

The exercise checks the HTML structure. The media files do not need to be available for the markup to be correct.

Section C: Semantic HTML

Exercise 9: Replace generic containers where the role is clear

The following page uses `<div>` for every major area:

```
<body>
  <div>
    <h1>Community courses</h1>
    <p>Short courses for local learners.</p>
  </div>

  <div>
    <a href="#courses">Courses</a>
    <a href="#contact">Contact</a>
  </div>

  <div>
```

```
    <div id="courses">
      <h2>Weekend courses</h2>
      <p>Choose from three beginner sessions.</p>
    </div>
  </div>

  <div id="contact">
    <p>Contact the course team.</p>
  </div>
</body>
```

Replace suitable containers with:

```
header
nav
main
section
footer
```

Apply these rules:

- The introductory page content belongs in `<header>`.
- The navigation links belong in `<nav>`.
- The primary page content belongs in `<main>`.
- The weekend course content belongs in `<section>`.
- The contact information belongs in `<footer>`.
- Do not replace an element merely because a semantic element exists somewhere in HTML.

Exercise 10: Use `section` and `article` for different roles

A page contains a collection of independent workshop announcements.

Create HTML with:

- One `<section>` for the complete collection
- The heading `Upcoming workshops`
- Three `<article>` elements
- One article for each workshop

Use this information:

```
HTML foundations
Saturday, 10 AM
Learn how to structure a web page.

Responsive CSS
Sunday, 10 AM
```

```
Adapt layouts to different screen sizes.

JavaScript basics
Sunday, 2 PM
Add behaviour to a web page.
```

Each article should contain:

- A heading
- A time
- A short description

Then explain:

1. Why the collection uses `<section>`
2. Why each workshop uses `<article>`

Exercise 11: Improve a page that already renders

The following HTML is valid enough to display, but its structure does not describe the page clearly:

```
<body>
  <div>
    <h1>Learning centre</h1>
  </div>

  <div>
    <a href="#about">About</a>
    <a href="#courses">Courses</a>
  </div>

  <div id="about">
    <p>
      The centre offers weekend classes.
    </p>
  </div>

  <div id="courses">
    <p>
      HTML, CSS, and JavaScript courses
      are available.
    </p>
  </div>

  <div>
    Copyright 2026
```

```
  </div>
</body>
```

Improve the structure so that:

- The page title is inside `<header>`.
- The links are inside `<nav>`.
- The primary content is inside `<main>`.
- The `about` and `courses` areas use sections.
- Each section has a heading.
- The copyright information is inside `<footer>`.

Do not add CSS or change the meaning of the content.

Section D: Forms

Exercise 12: Connect a label to an input

Start with:

```
<label>Email address</label>
<input type="email">
```

Add attributes so that:

- The input has the ID `email`.
- The label refers to the same ID.
- The input has the name `email`.
- Clicking the label focuses the input.

Then create another correctly connected label and input for the participant's full name.

Use:

```
id="full-name"
name="fullName"
type="text"
```

Exercise 13: Choose the correct form control

Choose a suitable HTML form control for each requirement.

Use one of these for each answer:

```
input
textarea
select
radio
checkbox
```

Requirements:

1. Enter a full name on one line.
2. Enter an email address.
3. Write a question that may contain several sentences.
4. Choose exactly one session from three visible options.
5. Choose any number of topics of interest.
6. Choose one city from a longer list of cities.

After choosing the controls, write the corresponding HTML for requirements 3, 4, and 6.

Exercise 14: Create one radio-button group

Create a form section titled `Preferred session`.

Provide these choices:

```
Saturday morning
Saturday afternoon
Sunday morning
```

Apply these rules:

- Each option uses `type="radio"`.
- Each radio input has a unique ID.
- Each label is connected to its input.
- Every radio input uses the same name: `preferredSession`.
- Each radio input has a useful value.
- Only one option can be selected at a time.

After writing the markup, explain why the radio buttons must share the same `name`.

Exercise 15: Create independent checkboxes and a select menu

Create a form section titled `Workshop preferences`.

Topics of interest

Allow participants to choose any number of these topics:

```
HTML
CSS
JavaScript
```

Each checkbox should have:

- A unique ID
- A connected label
- The name `topics`
- A value that identifies the selected topic

Experience level

Create a `<select>` element with:

```
Choose a level
Beginner
Intermediate
```

Apply these rules:

- The select element uses `id="experience-level"`.
- Its name is `experienceLevel`.
- A connected label describes the field.
- Each choice uses an `<option>`.

Then explain why checkboxes suit the topics while a select menu suits the experience level.

Exercise 16: Repair a registration form

The following form contains several problems:

```
<form>
  <label for="participant-name">
    Full name
  </label>

  <input
    id="name"
    type="text"
  >

  <label for="email">
    Email address
  </label>

  <input
```

```
    id="email"
    type="email"
  >

  <p>Preferred session</p>

  <input
    id="morning"
    type="radio"
    name="morning-session"
    value="morning"
  >

  <label for="morning">
    Morning
  </label>

  <input
    id="afternoon"
    type="radio"
    name="afternoon-session"
    value="afternoon"
  >

  <label for="afternoon">
    Afternoon
  </label>

  <label for="question">
    Your question
  </label>

  <input
    id="question"
    type="text"
  >
</form>

<button type="submit">
  Register
</button>
```

Correct the form so that:

- Every label matches the ID of its control.
- Both radio buttons belong to one group.
- The question uses a `<textarea>`.
- Every successful form field has a meaningful `name`.
- The submit button is inside the form.
- All IDs remain unique.

Do not add new fields.

Final exercise: Build a community workshop page

Create this file:

```
html-final/
└── index.html
```

Build a complete HTML document for a community workshop page.

Do not add CSS or JavaScript.

Part 1: Document structure

Include:

- `<!DOCTYPE html>`
- `<html lang="en">`
- UTF-8 character encoding
- A viewport meta element
- The title `Community web workshop`
- A visible `<body>`

Part 2: Page header and navigation

Create a `<header>` containing:

- The page heading `Community web workshop`
- A short paragraph describing it as a beginner workshop

Create a `<nav>` containing links to:

```
#overview
#sessions
#registration
```

Use meaningful link text.

Part 3: Main workshop content

Create a `<main>` containing these sections.

Overview

Use:

```
<section id="overview">
```

Include:

- A section heading
- A paragraph explaining that the workshop covers HTML, CSS, and JavaScript
- An image using `./images/web-workshop.jpg`
- Useful alternative text
- An unordered list of the three topics

Sessions

Use:

```
<section id="sessions">
```

Include two independent workshop articles:

```
Morning session
Saturday, 10 AM
HTML structure and semantic elements

Afternoon session
Saturday, 2 PM
Forms, links, images, and media
```

Each session should use an `<article>` with a heading and supporting text.

Media preview

Add a video with:

```
./media/workshop-preview.mp4
```

Include controls, a source type, and fallback text.

Part 4: Registration form

Use:

```
<section id="registration">
```

Add a heading and a form containing:

Full name

- Text input
- Connected label
- ID `full-name`
- Name `fullName`

Email address

- Email input
- Connected label
- ID `email`
- Name `email`

Preferred session

Use one radio group with:

```
Morning
Afternoon
```

The radio buttons should share the name `preferredSession`.

Topics of interest

Use independent checkboxes for:

```
HTML
CSS
JavaScript
```

Use the name `topics` for the checkboxes.

Experience level

Create a select menu containing:

```
Choose a level
Beginner
Intermediate
```

Question

Add a textarea for an optional question.

Submission

Add a submit button inside the form with the text:

```
Register for the workshop
```

Part 5: Footer

Add a `<footer>` containing:

```
Contact the community learning team for support.
```

Completion checks

Confirm that:

- The document has one `<h1>`.
- Visible content is inside `<body>`.
- Navigation links point to existing IDs.
- Heading levels describe the page hierarchy.
- The image contains useful alternative text.
- The topic list uses `<ul>`.
- Each session is an `<article>`.
- The video includes controls and fallback text.
- Every form label is connected to its control.
- Radio buttons form one group.
- Checkboxes allow multiple selections.
- The longer question uses a textarea.
- The submit button is inside the form.
- IDs are unique.
- No CSS, Tailwind CSS, or JavaScript has been added.